

Unidad 2

Programación de hilos

2.4 Ejecutores de hilos

Programación de servicios y procesos

Curso 2025/2026

Contenido

- ▶ Recapitulación
- ▶ Ejecutores de hilos
- ▶ Callables y Futures

Recapitulación

- ▶ Hemos visto hasta ahora:
 - ▶ La clase Thread
 - ▶ El interfaz Runnable
 - ▶ La protección de secciones críticas usando synchronized
 - ▶ Los métodos wait y release para comunicar procesos productores y consumidores que acceden a secciones críticas

Clase Thread

- ▶ Se crean hilos heredando de ella y sobrescribiendo su método "run"

```
public class MiHilo extends Thread {  
    ...  
    @Override  
    public void run(){  
        // Código a ejecutar por el hilo  
    }  
}
```

- ▶ Se lanza el hilo llamando al método start

```
Thread t = new MiHilo();  
t.start();
```

Interfaz runnable

- ▶ Se define una clase que implementa la interfaz Runnable e incorpora el código a ejecutar en su método “run”.

```
public class MiTarea implements Runnable {  
    ... // atributos, constructores, métodos  
    @Override  
    public void run() {  
        // Código a ejecutar por el hilo  
    }  
}
```

- ▶ Se ejecuta un hilo creando una instancia de thread con el objeto runnable como parámetro

```
MiTarea tarea = new MiTarea()  
Thread t = new Thread(tarea);  
t.start();
```

Implementar Runnable vs extender Thread

- ▶ El Runnable puede heredar de cualquier clase
- ▶ Separación de responsabilidades:
 - ▶ Runnable → qué se hace
 - ▶ Thread → cómo se ejecuta
- ▶ Un Runnable se puede ejecutar en distintos threads

```
Runnable tarea = new MiTarea();  
new Thread(tarea).start();  
new Thread(tarea).start(); // misma lógica, otro hilo
```

Concurrencia avanzada en Java

- ▶ Todo lo visto hasta ahora es de Java 1.1 (1997)
- ▶ Ahora avanzaremos hasta Java 5 (2005) cuando se creó la librería `java.util.concurrent`

Ejecutores de hilos

- ▶ Un ejecutor de hilos es un gestor de hilos
- ▶ Controla:
 - ▶ Cuantos hilos se ejecutan a la vez
 - ▶ Cuando dejan de funcionar
- ▶ Facilita (con objetos Callable y Future):
 - ▶ La obtención de resultados
 - ▶ El manejo de errores

Sin ejecutores de hilos (hasta ahora)

```
Thread[] hilos = new Thread[10];

for (int i = 0; i < 10; i++) {
    hilos[i] = new Thread(() -> {
        hacerTrabajo();
    });
    hilos[i].start();
}

// Esperar a que acaben
for (Thread t : hilos) {
    try {
        t.join();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}
```

Limitaciones de la ejecución manual

- ▶ Escalabilidad
 - ▶ El sistema operativo puede mantener más hilos que núcleos pero...
 - ▶ ¿qué pasa en aplicaciones con cientos de miles de hilos? (por ejemplo, servidores)
- ▶ Farragoso
 - ▶ Crear tareas en runnables
 - ▶ Crear hilos
 - ▶ Asociar hilos a runnables
 - ▶ Arrancarlos
- ▶ Los hilos no devuelven nada
 - ▶ Los valores de retorno se deben gestionar aparte
 - ▶ Si una tarea lanza una excepción: el hilo muere y nadie se entera

Con un ejecutor

```
ExecutorService executor = Executors.newFixedThreadPool(4);
```

```
for (int i = 0; i < 10; i++) {  
    executor.submit(() -> hacerTrabajo());  
}
```

```
executor.shutdown();  
executor.awaitTermination(1, TimeUnit.MINUTES);
```

- ▶ No hace falta crear hilos, solo pasar tareas al ejecutor
- ▶ El ejecutor mantiene un número limitado de hilos a la vez
 - ▶ Ahorro de memoria respecto a arrancarlos todos
- ▶ Espera conjunta a que acaben todos los hilos (awaitTermination)

Como crear un ejecutor

- Usar clase Executors

// Hilos segun se necesiten

```
ExecutorService executor = Executors.newCachedThreadPool();
```

// Número fijo de hilos

```
ExecutorService executor = Executors.newFixedThreadPool(N);
```

// Tantos hilos como procesadores

```
ExecutorService executor = Executors.newFixedThreadPool(  
    Runtime.getRuntime().availableProcessors());
```

// Un solo hilo

```
ExecutorService executor = Executors.newWorkStealingPool();
```

Métodos de ExecutorService

Método	Parámetro	Devuelve	Función
execute(...)	Runnable	-	Ejecutar
submit(...)	Runnable o callable	Objeto Future	Ejecutar y devolver resultado de tarea
shutdown()	-	-	Cerrar : no se aceptan más tareas (no bloquea)
awaitTermination(...)	timeout, unidades	booleano indicando si las tareas acabaron a tiempo	Esperar a que acaben las tareas con tiempo máximo
shutdownNow()	-	-	Forzar interrupción de tareas

Finalización

- ▶ El método shutdown cierra el lanzador de hilos (pero los hilos siguen)
- ▶ El método awaitTermination espera a que:
 - ▶ se cierre el lanzador de hilos
 - ▶ acaben todos los hilos
 - ▶ o a que se pase el tiempo
- ▶ Sin timeout, espera indefinidamente
- ▶ Manera de producir un cuelgue:

```
// executor.shutdown();  
executor.awaitTermination();
```

Finalización

- ▶ Forma estándar de acabar:

```
executor.shutdown();  
executor.awaitTermination(timeout, unit);
```

- ▶ ¡Pero la JVM no termina mientras haya hilos activos!
- ▶ Si queremos garantizar el cierre

```
executor.shutdown();  
if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {  
    // Timeout: forzar cierre  
    executor.shutdownNow();  
}
```

Callable

- ▶ Un Callable se parece a un Runnable: es una tarea que se ejecuta en un hilo
 - ▶ Interfaz funcional - define un único método
- ▶ Método call() en lugar de run():
 - ▶ devuelve un resultado de un determinado tipo
 - ▶ puede lanzar una excepción

```
public interface Callable<V> {  
    V call() throws Exception;  
}
```


Cómo se crea un callable

- ▶ Al igual que con Runnable, como clase o con una lambda

```
// Como clase
class Suma implements Callable<Integer> {
    @Override
    public Integer call() throws Exception {
        return 2 + 3;
    }
}
```

```
// Con lambda
Callable<Integer> tarea = () -> 2 + 3;
```

Ejecución de un callable

- ▶ No se ejecuta directamente
- ▶ Se usa submit() en un ExecutorService

```
ExecutorService executor = Executors.newSingleThreadExecutor();  
Future<Integer> future = executor.submit(tarea);
```

Objeto Future

- ▶ Interfaz genérica parametrizada con el tipo de resultado que devuelve Callable
- ▶ Representa:
 - ▶ Un resultado que llegará en el futuro
 - ▶ Una excepción si la tarea falla
- ▶ El resultado se obtienen con el método get()

```
Integer resultado = future.get(); // espera si aún no ha terminado
```

Excepciones en objetos Future

```
Callable<Integer> tarea = () -> {  
    if (Math.random() > 0.5) {  
        throw new IOException("Fallo en la tarea");  
    }  
    return 42;  
};  
  
Future<Integer> future = executor.submit(tarea);  
  
try {  
    Integer valor = future.get();  
} catch (ExecutionException e) {  
    System.out.println("La tarea falló: " + e.getCause());  
}
```

Espera no bloqueante

► Método isDone

```
Future<Integer> future = executor.submit(tarea);
```

```
// en algún punto del programa
if (future.isDone()) {
    Integer resultado = future.get(); // aquí ya no bloquea
}
```

► Espera con timeout

```
try {
    Integer resultado = future.get(200, TimeUnit.MILLISECONDS);
} catch (TimeoutException e) {
    System.out.println("Aún no ha terminado");
}
```

Espera delegada

- ▶ Es otro hilo el que espera el resultado

```
executor.submit(() -> {  
    try {  
        Integer resultado = future.get();  
        System.out.println("Resultado: " + resultado);  
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
});
```